Week 09 Zobrist Hashing and Transposition Tables

Project: Research Design and Implementation of a Gomoku AI Engine
Module: Basic Project
Report language: English
Execution date: 17 October 2026
Status: Implemented and executed native reference experiments with an identified cache reuse limitation
Evidence boundary: Local `caro-core-reference` library and the reproducible `research/week_05_09` harness. Browser and WebAssembly integration were not executed.

1 Abstract

Different legal move sequences can reach the same Gomoku position. A transposition table can reuse prior search work, but its key and stored value must describe the same search problem. This week examined the existing 64 bit Zobrist implementation, incremental updates, depth qualified Alpha Beta bounds, replacement behavior, and controlled comparisons against plain Alpha Beta. The work added an executable research harness.

[A] Incremental hashing matched recomputation in 3,998 forward and 3,998 undo transitions, with four additional terminal winning axes. Across the complete search matrix, 208 cold table score comparisons passed. In a bounded clustered fixture at depth four, plain Alpha Beta visited 1,094 nodes and the cold table version visited 1,088, with four usable hits from 255 counted lookups. Median local duration increased from 774 to 812 microseconds. Repeating the same root with its warm table required nine nodes, but reusing that table after changing the root player produced score 19,800 where a fresh table returned zero. Consequently, the completed work establishes bounded native correctness and exposes a concrete cache scope defect; it does not establish safe cross turn reuse or a universal speed improvement.

2 Objectives and Research Questions

The objective was to test cache validity against the actual source. The questions were:

1. Does incremental XOR updating agree with a full board hash after play, terminal play, and undo?

2. Does a cold transposition table preserve the plain Alpha Beta value on the same configured tree?

3. What node reduction, usable hit rate, and local timing occur in the bounded fixture corpus?

4. Which state and search assumptions must remain fixed when a table is reused?

Acceptance required parity assertions, root restoration, preserved raw records, and explicit failure reporting. The project convention uses [A] for source or executed evidence, [B] for primary literature, [C] for inference, and [D] for recommendations.

3 Background and Mathematical Model

3 1 Transpositions

A transposition is an identical position reached through different legal action orders. With alternating players, the same player's stones can be exchanged in the history while the final placement and side to move agree. Such histories must also remain legal and nonterminal before reaching the position. Equal

board placement alone does not justify equating positions governed by different rules or evaluation perspectives.

[B] Zobrist introduced a method that combines random feature codes to support game position lookup. The original technical report also addresses retrieval errors, so its use should not be described as a collision proof identity scheme. See Zobrist 1970 University of Wisconsin Technical Report 88.

3 2 Hash Definition

Let $Z_{i,p}$ be a deterministic pseudo random 64 bit key for player p occupying board index i, and let Z_t be the side key. Empty cells contribute nothing. The checked implementation computes:

$$H(s) = \bigoplus_{i:B_i \neq 0} Z_{i,B_i} \oplus T(s)$$

where $T(s) = Z_t$ when O is to move and zero otherwise. There are 225×2 piece keys plus one side key. These are produced by SplitMix64 from the fixed experimental seed 20,261,017. This seed supports repeatability; it does not provide cryptographic security.

After move m by player p, the correct update follows the actual state transition:

$$H(s') = H(s) \oplus Z_{m,p} \oplus \Delta_t$$

Here $\Delta_t = Z_t$ if the side changes, and zero otherwise. In `GameState::play`, a winning move leaves `to_move` equal to the mover. Therefore an unconditional side toggle gives the wrong key on terminal wins. The harness deliberately verifies this negative case on all four winning axes.

XOR is self inverse, since $x \oplus a \oplus a = x$. Undo can therefore apply the same piece and actual side deltas, provided the board and state metadata are also restored. Hash equality does not by itself prove that `last_move`, status, count, or rule configuration were restored. The separate root restoration assertions compare the complete debug representation of `GameState`, including those fields.

3 3 Collision Risk

[C] Under an ideal uniform 64 bit model, the approximate probability of at least one collision among n distinct keys is:

$$P \approx 1 - \exp\left(-\frac{n(n-1)}{2 \cdot 2^{64}}\right)$$

For one million distinct positions the approximation is about 2.71×10^{-8}; at one billion it is about 2.67 percent. These are analytical estimates, not measured failure rates. The Rust map verifies its complete `u64` key against ordinary bucket collisions, but two board positions with the same Zobrist signature remain indistinguishable. This implementation stores no second signature or full board verification payload.

4 Bound Semantics and Cache Validity

[B] Alpha Beta can return the minimax value without exploring every sibling when an inherited bound proves the sibling irrelevant. Its correctness and its sensitivity to search order are established in Knuth and Moore 1975 An Analysis of Alpha Beta Pruning.

For a fixed depth, evaluator, rule profile, candidate policy and root perspective, write $V_d(s)$ for the configured tree value. A table entry contains remaining depth, score, bound and optional best move. Its logical meaning is:

Bound	Meaning	Probe action
Exact	$V_d(s) = v$	Return the score
Lower	$V_d(s) \geq v$	Raise alpha
Upper	$V_d(s) \leq v$	Lower beta

If the narrowed window satisfies $\alpha \geq \beta$, the stored bound can cause an early return. An entry whose depth is insufficient still provides a move ordering hint. The source classifies a newly searched result using the original window: a value at or below original alpha becomes an upper bound; at or above original beta it becomes a lower bound; otherwise it becomes exact.

[A] The source accepts an entry when `entry.depth >= depth`. This is a common search policy, but a deeper horizon value is not mathematically identical to a shallower heuristic horizon value. The differential experiment uses a cold table per configuration, avoiding claims that arbitrary deeper warm entries preserve an exact requested finite horizon. The public root API supplies the full integer window; independent narrow window injection was not tested.

The table key omits rule settings, evaluation weights, candidate radius and cap, and root evaluation perspective. Its scores are relative to the player at the search root. The side to move in a board key cannot replace that root perspective: an identical internal position can be evaluated for a different root player in a later turn. Fresh per search tables are therefore essential to the scope validated here.

Terminal search scores additionally depend on root ply:

$$S_{win} = M - ply, S_{loss} = -M + ply$$

with $M = 10,000,000$. [A] The implementation stores these values without mate distance normalization. [D] Reuse across different root plies requires a defined mate range and normalization: add storage ply to positive mate scores and subtract it from negative scores; reverse those adjustments at retrieval. This is a proposed repair contract, not an implemented or verified change. A threshold must also distinguish terminal scores from heuristic scores near the allowed clamp.

5 Source Evidence and Responsibility

File	Responsibility	Checked behavior	Confidence
`src/zobrist.rs`	Position signature	Piece keys and actual side delta	High
`src/state.rs`	Play and undo	Terminal turn stays with mover	High
`src/search/optimized.rs`	Cached search	Depth and bound probe plus store	High
`src/search/transposition.rs`	Entry storage	HashMap and depth preference	High
`src/search/iterative.rs`	Repeated depth search	Shared table within one root	High

All source paths in this table are relative to `caro-core-reference`. The new runner owns fixture construction, timing, assertions and exported records; it does not redefine the engine evaluator or search functions.

[A] `TranspositionTable` is `HashMap<u64, TTEntry>`. Its capacity argument reserves space and does not impose a memory limit. Shallower entries cannot replace deeper entries; equal or deeper entries can replace them. There is no generation field, aging policy, fixed bucket count, or eight megabyte

slab. `size_of::<TTEntry>()` was 12 bytes on the checked native toolchain, excluding map keys, buckets, slack and allocator overhead. The earlier proposed sixteen byte layout and fixed memory claims are not descriptions of this source.

6 Experimental Method

The environment was Linux x86 64, Intel Xeon W 11855M, Rust 1.98.1 and Cargo 1.98.1, in release profile. The retained run began at 01:16:23 UTC on 17 October 2026. Full timestamps, environment details, source file hashes, and commands appear in `research/week_05_09/results/provenance.json`. Its combined source digest is `4625f3ba6b47e6d30723b0eb89ec8e28672198be095cee50db5ecd3fbc58a415`.

Four controlled fixtures cover a center stone, a four stone cluster, an immediate winning line, and an opponent threat. Sixteen additional positions are generated by legal play using seed 5,006,070,809. Tactical fixtures are intentionally synthetic; they are not represented as sampled participant games. The matrix contains all legal and radius one uncapped search at depths one and two, radius one cap twelve at depths one through three, and radius one cap eight at depth four. The capped depth four case exposes interior transpositions.

Each controlled configuration has three sequential repeats. Timings exclude cloning and hash/table setup. CPU pinning, isolated host load and significance analysis were not performed. Minima, medians and maxima are retained. No browser, Worker, Wasm, mobile device, or production clock is involved.

The optimized API has no independent hashing off or table off option. The valid comparison is plain Alpha Beta against cold optimized Alpha Beta with otherwise matching configuration. Every measured cold call starts with a new table. Requested natural ordering does not categorically disable ordering: a cached best move can invoke the existing ordering function internally.

7 Results

7 1 Executed Correctness Gates

[A] The retained reference test log records six unit and three integration tests passing. The experiment records 544 measured rows and 224 summaries. It executed 546 counted search calls including the cap diagnostic, and 551 root restoration checks including direct cache and budget probes. There were 112 Minimax versus Alpha Beta value and move comparisons, 112 ordering value comparisons, and 208 cold table value comparisons, all passing.

Hash testing passed 3,998 forward and 3,998 reverse transitions across one hundred seeded games. Some games terminate before the forty ply limit. Four explicit terminal winning axes also passed. The table tests roundtrip exact, lower and upper entries and verify shallower, deeper and equal depth replacement. Inspection of one bounded search found 48 exact, 34 lower and 170 upper entries. Observing those classes and passing the differential corpus does not prove every possible bound interaction or exclude hash collisions.

7 2 Cold Table Comparison

The following rows use radius one, cap eight, depth four, and requested natural order. Scores matched in each pair. Times are native median microseconds across three repeats.

Fixture	Plain nodes	TT nodes	Hits per lookups	Time plain to TT
Center	838	826	4 per 182	533 to 558
Clustered	1094	1088	4 per 255	774 to 812

Fixture	Plain nodes	TT nodes	Hits per lookups	Time plain to TT
Win	1766	1743	10 per 350	1466 to 1467
Block	1220	1220	0 per 261	901 to 960

For the clustered row, the substituted calculations are:

$$R_{nodes} = 1 - \frac{1088}{1094} = 0.548\,\%$$

$$R_{hit} = \frac{4}{255} = 1.569\,\%, S_{time} = \frac{774}{812} = 0.953$$

[A] The cold table reduced a few nodes without a measured timing improvement in these rows. At depths one through three the cold matrix recorded zero usable hits; its shallow interior structure and the implementation's absence of leaf probes explain that observation. A zero denominator at depth one means the hit ratio is undefined, not zero percent evidence of ineffective caching. These outcomes do not predict deeper, larger, or differently ordered searches.

7 3 Warm Reuse and Reproduced Scope Failure

[A] In the clustered depth four scope probe, repeating the same root with its existing table preserved score zero and visited nine nodes with eight counted usable hits. This is a warm cache result, separate from the cold matrix. The root uses a stored move hint without counting that probe in `tt_lookups`; the reported hit ratio concerns internal counted probes.

After playing the selected root move, the harness searched the resulting position at depth three. A fresh table returned zero; the previous root's table returned 19,800. The board state was restored after both searches. This reproducible mismatch is retained in `cache_scope_probe.json`.

The immediate cause is the acceptance of matching board keys whose stored scores belong to the preceding root player's evaluation. The system cause is caller managed table lifetime without an enforced evaluation namespace. The root cause is an incomplete cache validity contract. [D] Until a repair is implemented and tested, begin each new root search with a fresh table. Alternatives include tagging root perspective and configuration or adopting a consistently defined perspective independent score representation. The current task records this defect without silently patching the existing library.

7 4 Budget and Candidate Diagnostics

[A] A ten node direct budget abort restored the state. A fifty node iterative probe also restored state and retained a completed depth two result. Its aggregate reports forty completed iteration nodes and omits work inside the aborted iteration; it must not be read as total attempted cost. Separate candidate evidence demonstrates that a row major cap of six omits both immediate winning endpoints: the uncapped depth one score is 9,999,999 and the capped score is 99,990. A table cannot repair moves excluded by the candidate policy.

8 Limitations and Follow Up

The corpus validates selected finite trees and state transactions, not complete Gomoku strategy or engine strength. The evaluator recognizes maximal contiguous patterns and does not model every broken threat. Cold cache score parity does not establish safe warm reuse under changed player, rules, weights, cap, depth semantics, or mate ply. There was no fixed memory experiment, collision injection test, persistence test, browser memory profile, or native to Wasm parity run.

[D] The next implementation priority is a cache namespace contract with explicit root and configuration ownership, regression coverage for the reproduced mismatch, and a documented mate distance representation. Subsequent work can introduce an actually bounded table and measure replacement behavior. Performance evaluation should then use a larger declared corpus, controlled process conditions and separately measured adapter costs. Reported depth six latency and an eight megabyte budget require new executed evidence before they can be claimed.

9 Traceability and Reproduction

Question	Artifact	Outcome
Incremental consistency	`verification.json` and runner	Passed bounded transitions
Cold value equivalence	`search_runs.csv`	208 score pairs passed
Cache utilization	`search_summary.csv`	Small depth four reductions
Reuse validity	`cache_scope_probe.json`	Cross root mismatch reproduced
Storage contract	`tt_semantics.json`	Three bounds and depth replacement

Artifacts are in `research/week_05_09/results`. Reproduce with `python3 research/week_05_09/run.py`; it retains logs, source hashes and raw records. Reruns overwrite results. Game integration requires separate runtime verification.